# Week 1-2 – Foundations of Machine Learning and Probability

## Core Concepts of the Lecture and Material

### What is Machine Learning?

Machine learning enables computers to learn from data and experience, similar to how humans learn. Think of it like training a new employee - you show them examples of good work (training data), and they learn patterns to handle new situations (test data). The core technology behind AI systems like ChatGPT, machine learning uses mathematics, statistics, and algorithms to build models that find relationships between inputs and outputs.

### Supervised Learning

This is like having a teacher supervise your learning. You're given input-output pairs where the correct answers are provided. There are two main types:

- **Regression**: Predicting continuous values (like Olympic winning times or stock prices)
- **Classification**: Predicting categories (like cat vs. dog images or spam vs. not-spam emails)

### The Machine Learning Process

Imagine you're an investor trying to predict stock prices. You have historical data (training set) showing how various factors affected past prices. You want to find a mathematical relationship (model) that can predict future prices (test set). The process involves:

1. Choosing a model type (like choosing between linear vs. complex investment strategies)
2. Finding the best parameters for that model (optimizing your strategy)
3. Testing how well it generalizes to new data (seeing if your strategy works on new stocks)

### Models and Parameters

A model is simply a mathematical function that maps inputs to outputs. For example, a polynomial model might look like: output = $w_0 + w_1x + w_2x^2 + \ldots$ The w's are parameters we need to learn from data. Think of these as dials you adjust to make your prediction model more accurate.

### Training and Error Minimization

Training is like tuning your investment strategy to minimize losses. We define an error function that measures how wrong our predictions are, then adjust parameters to minimize this error. Just as you'd analyze past investment mistakes to improve, the model learns from its training errors.

### Overfitting and Underfitting

- **Underfitting**: Your model is too simple, like using only last month's data to predict complex stock trends
- **Overfitting**: Your model is too complex, like memorizing exact past stock movements instead of learning general patterns

### Regularization

This prevents overfitting by penalizing complex models, similar to how experienced investors avoid over-complicated strategies that might work perfectly on past data but fail with new investments. We add a penalty term that discourages extreme parameter values.

### Model Selection and Cross-Validation

Since we can't use test data for training (that's cheating), we use techniques like k-fold cross-validation. Imagine splitting your historical stock data into 5 chunks, training on 4 chunks and testing on the 1 remaining chunk, then rotating through all combinations to find the best model complexity.

### Probability and Uncertainty

Real-world data is noisy, like stock prices fluctuating due to random market events. Probability theory helps us quantify and manage this uncertainty. There are two main approaches:

- **Frequentist**: Believes there's one true model (like believing there's one "correct" stock valuation)
- **Bayesian**: Treats models as uncertain and updates beliefs as new data arrives (like adjusting your investment thesis as new market information emerges)

### Maximum Likelihood Estimation

This principle says: choose the model parameters that make your observed data most probable. If you saw a stock rise 8 out of 10 days, the most likely probability of it rising is 80%.

### Bayesian Approach

This method combines prior beliefs with new evidence. Imagine you think a stock has 60% chance of success (prior), then see it succeed 9 out of 10 times. Bayesian updating combines these to give a refined estimate (posterior).

### Bootstrap Method

When you can't collect more data, bootstrap creates virtual datasets by sampling with replacement from your original data. It's like analyzing multiple hypothetical market scenarios based on your existing data to understand how stable your predictions are.

## Key Formulae

### 1. Polynomial Model

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

- $x$: input variable (e.g., Olympic year)
- $w_j$: model parameters/coefficients
- $M$: degree of polynomial (model complexity)
- $y$: predicted output

*Example*: For a linear model (M=1) predicting winning time: $y = w_0 + w_1 \times \text{year}$

### 2. Sum of Squares Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} [y(x_n, \mathbf{w}) - t_n]^2$$

- $t_n$: actual target value for n-th data point
- $N$: number of training data points
- The $\frac{1}{2}$ makes derivatives cleaner

*Example*: If predicted time is 9.8s but actual is 9.9s, error = (9.8-9.9)² = 0.01

### 3. Regularized Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} [y(x_n, \mathbf{w}) - t_n]^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

- $\lambda$: regularization parameter controlling penalty strength
- $\|\mathbf{w}\|^2 = w_0^2 + w_1^2 + \ldots + w_M^2$: sum of squared parameters

*Example*: With λ=0.1 and weights [1, 2, 3], penalty = 0.1 × (1+4+9)/2 = 0.7

### 4. Root Mean Square (RMS) Error

$$E_{RMS} = \sqrt{\frac{2E(\mathbf{w}^*)}{N}}$$

- $E(\mathbf{w}^*)$: minimum error achieved
- Used to compare models across different dataset sizes

*Example*: If total error=5 with N=10 points, RMS = √(2×5/10) = √1 = 1.0

### 5. Coin Likelihood Function

$$p(\mathcal{D}|\omega) = \omega^H(1-\omega)^T$$

- $\omega$: probability of heads
- $H$: number of heads observed
- $T$: number of tails observed
- $\mathcal{D}$: observed data

*Example*: With 3 heads and 7 tails, likelihood = $\omega^3(1-\omega)^7$

### 6. Maximum Likelihood for Coin

$$\omega_{ML} = \frac{H}{H+T}$$

- The intuitive result: proportion of heads observed

*Example*: 3 heads out of 10 tosses gives $\omega_{ML} = 3/10 = 0.3$

### 7. Beta Distribution (Bayesian Prior)

$$p(\omega) \propto \omega^{a-1}(1-\omega)^{b-1}$$

- $a, b$: hyperparameters controlling prior belief
- Used to represent uncertainty about coin probability

*Example*: Beta(2,2) represents belief that coin is likely fair (centered at 0.5)

### 8. Bayesian Posterior

$$p(\omega|\mathcal{D}) \propto \omega^{a+H-1}(1-\omega)^{b+T-1}$$

- Combines prior belief (a,b) with observed data (H,T)
- Posterior is also a Beta distribution: Beta(a+H, b+T)

*Example*: Prior Beta(2,2) with 3 heads, 7 tails gives posterior Beta(5,9)

### 9. Bayesian Prediction

$$p(\text{head}|\mathcal{D}) = \frac{a+H}{a+H+b+T}$$

- Predicts probability of next head using entire posterior distribution

*Example*: With prior Beta(2,2) and 3 heads, 7 tails: p(head) = (2+3)/(2+3+2+7) = 5/14 ≈ 0.357

## Likely Exam Questions

1. **Explain the difference between supervised and unsupervised learning with examples.**

- *Answer*: Supervised learning uses labeled data (input-output pairs) like predicting house prices from features (regression) or classifying emails as spam/not-spam (classification). Unsupervised learning finds patterns in unlabeled data, like grouping customers by purchasing behavior (clustering). (Lecture 1 & Textbook Section 1)

2. **What is overfitting and how can regularization help prevent it?**
   - *Answer*: Overfitting occurs when a model becomes too complex and fits training data noise instead of underlying patterns, leading to poor generalization. Regularization adds a penalty term to the error function that discourages large parameter values, effectively simplifying the model. (Lecture 1 & Textbook Section 2)
3. **Calculate the maximum likelihood estimate for a coin that shows heads 8 times in 12 tosses.**
   - *Answer*: Using $\omega_{ML} = H/(H+T) = 8/(8+4) = 8/12 = 2/3 \approx 0.667$. The coin has 66.7% probability of heads according to maximum likelihood. (Lecture 2 & Textbook Section 4)
4. **Explain why we need both training and test sets in machine learning.**
   - *Answer*: The training set is used to learn model parameters, while the test set evaluates generalization to unseen data. Using the same data for both would be cheating and wouldn't reveal overfitting. (Lecture 1 & Textbook Section 2)
5. **A polynomial of degree 9 fits training data perfectly but performs poorly on test data. Is this overfitting or underfitting? Explain.**
   - *Answer*: This is overfitting. The high-degree polynomial is too complex and has likely learned noise in the training data rather than the true pattern, so it fails to generalize. (Lecture 1 & Textbook Section 2)
6. **Describe how k-fold cross-validation works and why it's useful.**
   - *Answer*: The dataset is split into k equal folds. Each fold serves as validation set once while the other k-1 folds train the model. The average validation error across all folds estimates test error. This uses all data for both training and validation while maintaining honest evaluation. (Lecture 1 & Textbook Section 2)
7. **Given prior Beta(3,3) and data with 5 heads and 1 tail, find the posterior distribution.**
   - *Answer*: Posterior = Beta(3+5, 3+1) = Beta(8,4). The prior represented belief centered at 0.5, and data shifted belief toward higher probability of heads. (Lecture 2 & Textbook Section 4)
8. **What is the fundamental difference between frequentist and Bayesian statistics?**
   - *Answer*: Frequentists treat parameters as fixed unknown values to be estimated from data. Bayesians treat parameters as random variables with probability distributions representing uncertainty. (Lecture 2 & Textbook Section 4)
9. **Compute the error for a linear model y=2x+1 when x=3 and actual value t=8.**
   - *Answer*: Predicted y = 2×3+1 = 7. Error = $(7-8)^2$ = 1. With the 1/2 factor from the error function, contribution = 0.5. (Lecture 1 & Textbook Section 2)

10. **How does the bootstrap method help quantify uncertainty in machine learning?**

- *Answer*: Bootstrap creates multiple virtual datasets by sampling with replacement from original data. Training models on these different datasets shows how much estimates vary, quantifying uncertainty about parameters and predictions. (Lecture 2 & Textbook Section 4)

# Week 3-4 – Linear Regression and the Bias-Variance Trade-off

## Core Concepts of the Lecture and Material

### Linear Basis Function Models

Linear regression starts with the simplest model: $y = w^T x$, where we try to find weights that make predictions align with target values. But to handle more complex patterns, we extend this to non-linear basis functions: $y = w^T \varphi(x)$. Think of $\varphi(x)$ as transforming your raw data into more meaningful features—like how financial analysts transform raw stock prices into moving averages and volatility measures to better predict market trends. The basis function vector $\varphi(x)$ includes a dummy function (usually set to 1) and other non-linear transformations of input data.

### Learning Model Parameters

To find the best weights, we minimize an error function. The most common approach uses gradient descent: we calculate the direction of steepest descent (the gradient) and take small steps in that direction. Imagine trying to find the lowest point in a valley while blindfolded—you feel the slope beneath your feet and take small steps downhill. The learning rate controls how big these steps are; too large and you might overshoot the minimum, too small and it takes forever to converge. Stochastic gradient descent (SGD) improves efficiency by using just one data point at a time to estimate the gradient, making it like checking the slope under just one foot at a time rather than surveying the entire landscape.

### Regularization

Regularization prevents overfitting by adding a penalty term to the error function: total error = data-dependent error + $\lambda \times$ regularization function. Think of it like a financial regulator imposing capital requirements on banks—too much risk-taking (complexity) gets penalized. Ridge regression (L2 regularization) adds the squared magnitude of coefficients as penalty, encouraging small but non-zero weights. LASSO (L1 regularization) adds the absolute value of coefficients, which can drive some weights to exactly zero (feature selection). The regularization parameter $\lambda$ controls this trade-off: high $\lambda$ means simpler models, low $\lambda$ means more complex models that fit training data closely.

### Bias-Variance Trade-off

This is the heart of model generalization. Generalization error decomposes into three parts: bias² + variance + irreducible error. Bias represents systematic errors from overly simplistic

assumptions (like always predicting the stock market will go up). Variance represents errors from being too sensitive to training data noise (like making investment decisions based on a single day's market fluctuations). The trade-off is fundamental: as model complexity increases, bias decreases but variance increases. The optimal model balances these two components.

### Diagnosing Model Problems

If your model has high test error, bias-variance analysis tells you how to fix it. High bias (underfitting) means your model is too simple—it doesn't capture the underlying pattern. This is like using only yesterday's closing price to predict today's market. Solutions: add more features, use more complex models, or reduce regularization. High variance (overfitting) means your model is too complex—it memorizes noise in the training data. This is like building an overly complicated trading algorithm that perfectly predicts past market movements but fails on new data. Solutions: get more training data, remove features, or increase regularization.

### Bias-Variance Visualization

The lectures show concrete examples where different model complexities (like polynomial degrees or regularization strengths) affect bias and variance. With simple models (low complexity), all learned functions are similar (low variance) but far from the true function (high bias). With complex models (high complexity), learned functions vary widely (high variance) but their average is close to the true function (low bias). The ideal model has both low bias and low variance.

## Key Formulae

### Linear Regression Model

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

- $\mathbf{w}$: weight vector (parameters to learn)
- $\boldsymbol{\phi}(\mathbf{x})$: basis function vector (includes $\phi_0(\mathbf{x}) = 1$)
- $M$: number of basis functions

*Example*: For a housing price prediction with features (size, bedrooms), using quadratic basis functions: $\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_2^2]^T$. If weights are $\mathbf{w} = [50, 0.1, 10, 0.001, 0.5]^T$ and house features are $[1000, 3]^T$, then predicted price = $50 + 0.1 \times 1000 + 10 \times 3 + 0.001 \times 1000^2 + 0.5 \times 3^2 = 50 + 100 + 30 + 1000 + 4.5 = 1184.5$ (thousand dollars).

### Sum-of-Squares Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2 = \frac{1}{2} \|\boldsymbol{\Phi}\mathbf{w} - \mathbf{t}\|^2$$

- $\mathbf{\Phi}$: design matrix (each row is $\phi(\mathbf{x}_n)^T$)
- $\mathbf{t}$: target vector
- $N$: number of training examples

*Example*: With 3 data points where predictions are [2.1, 3.9, 5.2] and targets are [2, 4, 5], error = 0.5×[(0.1)² + (-0.1)² + (0.2)²] = 0.5×(0.01+0.01+0.04) = 0.03.

### Regularized Error Function

$$E(\mathbf{w}) = E_D(\mathbf{w}) + \lambda E_W(\mathbf{w})$$

- $E_D(\mathbf{w})$: data-dependent error (e.g., sum-of-squares)
- $E_W(\mathbf{w})$: regularization function
- $\lambda$: regularization parameter (controls trade-off)

*Example*: If data error is 10.5 and regularization term is 2.3 with λ=0.5, total error = 10.5 + 0.5×2.3 = 11.65.

### Ridge Regression (L2 Regularization)

$$E(\mathbf{w}) = \frac{1}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

- $\|\mathbf{w}\|^2 = \sum_{j=0}^{M-1} w_j^2$: squared L2 norm of weights

*Example*: With weights [0.5, 1.2, -0.8], λ=0.1, regularization term = 0.1/2 × (0.25 + 1.44 + 0.64) = 0.05 × 2.33 = 0.1165.

### LASSO (L1 Regularization)

$$E(\mathbf{w}) = \frac{1}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2 + \lambda\|\mathbf{w}\|_1$$

- $\|\mathbf{w}\|_1 = \sum_{j=0}^{M-1} |w_j|$: L1 norm of weights

*Example*: With weights [0.5, 1.2, -0.8], λ=0.1, regularization term = 0.1 × (0.5 + 1.2 + 0.8) = 0.1 × 2.5 = 0.25.

### Bias-Variance Decomposition

$$\text{Generalization Error} = \text{bias}^2 + \text{variance} + \text{irreducible error}$$

$$\text{bias}(\mathbf{x}) = \mathbb{E}[\hat{y}(\mathbf{x})] - y(\mathbf{x})$$

$$\text{variance}(\mathbf{x}) = \mathbb{E}\left[(\hat{y}(\mathbf{x}) - \mathbb{E}[\hat{y}(\mathbf{x})])^2\right]$$

- $\hat{y}(\mathbf{x})$: prediction of model trained on specific dataset
- $\mathbb{E}[\hat{y}(\mathbf{x})]$: average prediction across different training datasets

- $y(\mathbf{x})$: true value

*Example*: If true value is 10, and model predictions across 3 datasets are [8, 12, 9], then:

- Average prediction = (8+12+9)/3 = 9.67
- Bias = 9.67 - 10 = -0.33
- Bias² = 0.11
- Variance = [(8-9.67)² + (12-9.67)² + (9-9.67)²]/3 = [2.79 + 5.43 + 0.45]/3 = 2.89

## Likely Exam Questions

1. **Explain the bias-variance trade-off in machine learning and how it relates to model complexity.**
   - *Answer*: The bias-variance trade-off describes how as model complexity increases, bias decreases (model fits training data better) but variance increases (model becomes more sensitive to noise in training data). Simple models have high bias but low variance, while complex models have low bias but high variance. The optimal model balances these to minimize total generalization error. As stated in the lecture transcript: "As model complexity increases, bias decreases but variance increases" and "The higher is the lander [regularization parameter], the more important is the regularisation term... so you are trying to learn a simpler model."
2. **When would you increase the regularization parameter λ in ridge regression, and why?**
   - *Answer*: You would increase λ when your model shows high variance (overfitting), meaning it performs well on training data but poorly on test data. Higher λ increases the penalty on large weights, forcing the model to be simpler and less sensitive to noise in the training data. As explained in the lecture: "The higher lander means less complex models, the lower lander means more complex model" and "When your model has a high variance... if you increase the data set size... you reduce the variance."
3. **Given a model with high test error, how would you determine if the problem is high bias or high variance?**
   - *Answer*: Compare training and test errors. High bias (underfitting) shows high training and high test error. High variance (overfitting) shows low training error but high test error. As stated in the lecture: "So let's assume that you have a model which has high error on a test set. And you ask yourself, do I need to train my model on a larger data set... When your model has a high variance... Probably that helps."
4. **Calculate the L2 regularization term for weights w = [0.2, -0.5, 0.8] with λ = 0.3.**
   - *Answer*: L2 regularization term = (λ/2) × ||w||² = (0.3/2) × (0.2² + (-0.5)² + 0.8²) = 0.15 × (0.04 + 0.25 + 0.64) = 0.15 × 0.93 = 0.1395. This comes directly from the ridge regression formula presented in the textbook section on regularization.

5. **What is the main difference between L1 (LASSO) and L2 (Ridge) regularization, and what practical effect does this difference have?**
   - *Answer*: L1 regularization uses the absolute value of weights ($||w||_1$), while L2 uses squared weights ($||w||^2$). The key practical difference is that L1 can drive some weights exactly to zero, performing feature selection, while L2 only shrinks weights toward zero. As noted in the textbook: "This particular choice of regulariser is known in the machine learning literature as weight decay because in sequential learning algorithms, it encourages weight values to decay towards zero, unless supported by the data."
6. **If a model has high bias, what are two practical steps you could take to improve its performance?**
   - *Answer*: 1) Add more features or use more complex basis functions to increase model flexibility. 2) Reduce regularization strength (decrease λ) to allow the model to fit the training data more closely. As explained in the lecture: "When your model has a high bias, usually this works very well... Your model is too simple. So by adding features you make it more flexible. And that helps."
7. **Explain why stochastic gradient descent might be preferred over batch gradient descent for large datasets.**
   - *Answer*: Stochastic gradient descent (SGD) processes one data point at a time, making it much faster per iteration and able to escape shallow local minima due to its inherent noise. For large datasets, batch gradient descent becomes computationally expensive as it requires processing all data points for each update. As described in the textbook: "In the case of the sum-of-squares error function, the stochastic gradient descent algorithm gives... This is known as the least-mean-squares (LMS) algorithm."
8. **Calculate the bias and variance for a model where the true value is 15, and predictions from 4 different training sets are [12, 18, 14, 16].**
   - *Answer*: Average prediction = (12+18+14+16)/4 = 15. Bias = 15 - 15 = 0. Variance = $[(12-15)^2 + (18-15)^2 + (14-15)^2 + (16-15)^2]/4 = [9 + 9 + 1 + 1]/4 = 20/4 = 5$. This follows the bias-variance decomposition formula presented in both the textbook and lecture transcripts.
9. **How does increasing the size of the training dataset affect bias and variance?**
   - *Answer*: Increasing training data size primarily reduces variance while having little effect on bias. More data helps the model generalize better by reducing sensitivity to noise in any particular training set. As stated in the lecture: "When your model has a high variance... if you increase the size of the data set... you reduce the variance. So your model... is complex for the data set that you have."
10. **Why can't the L1 regularization term be optimized using standard gradient descent, and what alternative is used?**

- *Answer*: The L1 regularization term ($||w||_1$) is not differentiable at zero, so standard gradient descent can't be applied directly. Instead, optimization algorithms based on sub-gradients

are used, which generalize the concept of gradients to non-differentiable functions. As explained in the textbook: "We note that the L1-regularisation term is not differentiable. Therefore, the training objective is not differentiable, hence it cannot be optimised using the (stochastic) gradient descent algorithm. There are optimisation algorithms though which can be used to optimise non-differentiable functions, such as algorithms based on the sub-gradient."

11. **In the context of linear regression with polynomial basis functions, how does increasing the polynomial degree affect bias and variance?**

- *Answer*: Increasing polynomial degree makes the model more complex, which decreases bias (the model can fit more intricate patterns) but increases variance (the model becomes more sensitive to noise in the training data). As shown in the lecture examples: "The simplest one is the zeroth order polynomial, then the 1st order, 3rd order, and then the 15th order is the most complex one... for high model complexity, we observed from this experiment that the variance is high among defeated functions, but the bias is low."

12. **If you have a model with high variance, would removing features help improve performance? Explain why or why not.**

- *Answer*: Yes, removing features would likely help. High variance indicates overfitting, where the model is too complex for the available data. Reducing features simplifies the model, decreasing its capacity to fit noise in the training data. As stated in the lecture: "it doesn't help [to remove features] when your model has high bias... but when your model has high variance, removing features reduces complexity and helps."

# Week 5-6 – Linear Models for Classification

## Core Concepts of the Lecture and Material

### Classification vs. Regression

Classification is about assigning inputs to discrete categories (like "pass" or "fail"), while regression predicts continuous values (like house prices). Think of classification like deciding whether to buy or sell a stock (discrete choices), while regression is like predicting the exact future price of that stock (a continuous value). In classification, we're trying to find the right "bucket" for each data point, whereas in regression we're trying to hit a precise numerical target.

### Linear Classification

Linear classification uses a simple mathematical function to separate data into classes. Imagine drawing a straight line (or a plane in higher dimensions) that divides your data points into different categories. The equation for this line is $y = w^T x + w_0$, where $w$ is a weight vector that

determines the orientation of the line, and $w_0$ is a bias term that shifts the line's position. For binary classification (two classes), we typically assign class +1 to points on one side of the line and class -1 to points on the other side.

### Perceptron Algorithm

The Perceptron is one of the simplest linear classification algorithms. It works like a trader adjusting their investment strategy based on mistakes. Imagine you're trading stocks and you make a wrong prediction—you'd adjust your strategy slightly to avoid that mistake next time. Similarly, the Perceptron starts with random weights and processes each training example one by one. If it misclassifies an example, it updates its weights to correct that mistake. The update rule is: $w^{(k+1)} = w^{(k)} + \eta t_n \phi_n$, where $\eta$ is a small learning rate (like how aggressively you adjust your trading strategy), $t_n$ is the true class label, and $\phi_n$ is the feature vector.

### Linear Separability

A problem is "linearly separable" if you can draw a straight line (or hyperplane) that perfectly separates all data points of different classes. Think of it like having two types of stocks that always move in opposite directions—you could draw a clear boundary between them. If data is linearly separable, the Perceptron is guaranteed to find a solution with zero training error, though it might take many iterations. However, many real-world problems aren't perfectly linearly separable, like stocks that sometimes behave similarly despite belonging to different sectors.

### Discriminative Models

Discriminative models directly learn the decision boundary between classes. They focus on "given this input, what's the probability it belongs to class A?" rather than modeling how the data was generated. Logistic regression is a prime example—it uses the sigmoid function to convert a linear function into a probability between 0 and 1. Think of this like a trader who only cares about whether a stock will go up or down tomorrow (the decision boundary), not about all the complex factors that might influence the stock price.

### Sigmoid Function

The sigmoid function ($\sigma(a) = \frac{1}{1+e^{-a}}$) is crucial for logistic regression. It takes any real-valued number and squishes it into a value between 0 and 1, perfect for representing probabilities. Imagine it like a stock volatility indicator that converts raw market data into a "probability of price increase" between 0% and 100%. The nice property of the sigmoid is that its derivative is simple: $\sigma'(a) = \sigma(a)(1 - \sigma(a))$, which makes optimization much easier.

### Maximum Likelihood Estimation

This is how we learn parameters for models like logistic regression. We construct a "likelihood" function that measures how probable our observed data is given certain model parameters, then find the parameters that maximize this likelihood. Think of it like finding the investment strategy that would have made the most money given past market data. For logistic regression,

we maximize the log-likelihood: $\ln p(t|w) = \sum_{n=1}^{N}\{t_n \ln y_n + (1-t_n)\ln(1-y_n)\}$, where $y_n$ is the predicted probability.

### Gradient Descent Optimization

Since we can't solve for logistic regression parameters directly, we use gradient descent. This is like a trader gradually adjusting their strategy by looking at which direction would most improve their returns. We calculate the gradient (derivative) of our error function with respect to each parameter, then take a small step in the opposite direction of the gradient. Stochastic gradient descent processes one data point at a time, making it efficient for large datasets.

### Probabilistic Generative Models

Unlike discriminative models, generative models try to understand how the data is generated. They model $P(x|C_k)$ (the probability of seeing features x given class $C_k$) and $P(C_k)$ (the prior probability of class $C_k$). Think of this like understanding the fundamental economic factors that generate stock price movements, rather than just predicting up/down. Using Bayes' theorem, we can then compute $P(C_k|x)$, which tells us the probability of a class given the observed features.

### Class Priors and Conditionals

For a binary classification problem, we need to model:

- Class priors: $P(C_1) = \phi$ and $P(C_2) = 1 - \phi$
- Class conditionals: $P(x|C_1)$ and $P(x|C_2)$

Often, we assume the class conditionals follow a Gaussian distribution. This means for each class, we need to estimate a mean vector and covariance matrix. The maximum likelihood estimates are simply the sample means and covariances of the data points in each class.

### Discriminative vs. Generative Models

Discriminative models (like logistic regression) directly model $P(C_k|x)$, while generative models (like Naive Bayes) model $P(x|C_k)$ and $P(C_k)$, then use Bayes' rule to get $P(C_k|x)$. Think of discriminative models as expert traders who've learned patterns for when to buy/sell, while generative models are like economists who understand the underlying market mechanics. Generative models typically require more data because they're modeling the full data distribution, not just the decision boundary.

## Key Formulae

### Perceptron Error Function

$$E(w) = -\sum_{n \in M} w^T \phi_n t_n$$

Where:

- $w$: weight vector
- $M$: set of misclassified examples
- $\phi_n$: feature vector for example $n$
- $t_n$: true class label for example $n$ (+1 or -1)

This error function measures how badly we're classifying the misclassified points. The negative sign ensures that when we minimize this error, we're actually improving classification.

*Example*: Suppose we have one misclassified point with $\phi_n = [2, 3]$, $t_n = 1$, and current weights $w = [-1, 1]$. Then $w^T \phi_n t_n = (-1)(2)(1) + (1)(3)(1) = 1$, so the error contribution is $-1$. If we update weights to $w = [0, 1]$, then $w^T \phi_n t_n = 3$, so error becomes $-3$—a lower (better) error.

### Perceptron Update Rule

$$w^{(k+1)} = w^{(k)} + \eta t_n \phi_n$$

Where:

- $w^{(k)}$: current weight vector
- $\eta$: learning rate (small positive value)
- $t_n$: true class label for example $n$
- $\phi_n$: feature vector for example $n$

This rule adjusts weights in the direction that would correct misclassifications.

*Example*: With $\eta = 0.1$, $t_n = 1$, $\phi_n = [2, 3]$, and current $w = [-1, 1]$, the update would be $w^{(new)} = [-1, 1] + 0.1 \times 1 \times [2, 3] = [-0.8, 1.3]$. This moves the decision boundary to better classify this point.

### Sigmoid Function

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

Where:

- $a$: input value (typically $w^T x$)

This function maps any real number to a value between 0 and 1, representing a probability.

*Example*: If $a = 2$, then $\sigma(2) = \frac{1}{1+e^{-2}} \approx 0.88$. If $a = -1$, then $\sigma(-1) = \frac{1}{1+e^{1}} \approx 0.27$.

### Derivative of Sigmoid

$$\sigma'(a) = \sigma(a)(1 - \sigma(a))$$

This simple derivative makes optimization efficient.

*Example*: If $\sigma(a) = 0.88$, then $\sigma'(a) = 0.88 \times (1 - 0.88) = 0.1056$.

### Log-Likelihood for Logistic Regression

$$\ln p(t|w) = \sum_{n=1}^{N} \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

Where:

- $t_n$: true label (0 or 1)
- $y_n = \sigma(w^T x_n)$: predicted probability
- $w$: weight vector
- $x_n$: feature vector for example $n$

This measures how well our model explains the observed data.

*Example*: For one data point with $t_n = 1$ and $y_n = 0.9$, the contribution is $1 \times \ln(0.9) + 0 \times \ln(0.1) \approx -0.105$. If $y_n = 0.6$, it's $\ln(0.6) \approx -0.511$—a worse (more negative) likelihood.

### Gradient of Log-Likelihood

$$\nabla_w \ln p(t|w) = \sum_{n=1}^{N} (t_n - y_n) x_n$$

Where:

- $t_n$: true label
- $y_n$: predicted probability
- $x_n$: feature vector

This tells us how to adjust weights to improve the model.

*Example*: For one point with $t_n = 1$, $y_n = 0.7$, $x_n = [2, 3]$, the gradient contribution is $(1 - 0.7) \times [2, 3] = [0.6, 0.9]$. We'd add this (scaled by learning rate) to our weights.

### Stochastic Gradient Descent Update

$$w^{(k+1)} = w^{(k)} + \eta(t_n - y_n) x_n$$

Where:

- $\eta$: learning rate
- $t_n$: true label
- $y_n$: predicted probability

- $x_n$: feature vector

This updates weights based on one data point at a time.

*Example*: With $\eta = 0.1$, $t_n = 1$, $y_n = 0.7$, $x_n = [2, 3]$, update is $w^{(new)} = w^{(old)} + 0.1 \times 0.3 \times [2, 3] = w^{(old)} + [0.06, 0.09]$.

### Gaussian Class Conditional

$$p(x|C_k) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma^{-1} (x - \mu_k)\right)$$

Where:

- $x$: feature vector
- $\mu_k$: mean vector for class $k$
- $\Sigma$: covariance matrix (assumed same for all classes)
- $D$: dimensionality of feature space

This models the probability of seeing features $x$ given class $C_k$.

*Example*: For a 1D feature ($D = 1$), with $\mu_k = 5$, $\sigma^2 = 4$ ($\Sigma = [4]$), and $x = 7$: $p(x|C_k) = \frac{1}{\sqrt{2\pi \times 4}} \exp(-\frac{(7-5)^2}{2 \times 4}) \approx 0.121$.

### Maximum Likelihood Mean Estimate

$$\mu_k = \frac{1}{N_k} \sum_{n \in C_k} x_n$$

Where:

- $N_k$: number of examples in class $k$
- $C_k$: set of examples in class $k$

This is simply the average of all examples in class $k$.

*Example*: For class 1 with points [2, 3], [4, 5], [6, 7], the mean is [(2+4+6)/3, (3+5+7)/3] = [4, 5].

### Maximum Likelihood Covariance Estimate

$$\Sigma = \frac{1}{N} \sum_{k=1}^{K} \sum_{n \in C_k} (x_n - \mu_k)(x_n - \mu_k)^T$$

Where:

- $N$: total number of examples

- $K$: number of classes
- $\mu_k$: mean of class $k$

This estimates the common covariance across all classes.

*Example*: With two classes, each with one point: class 1 [1, 2] with mean [1, 2], class 2 [3, 4] with mean [3, 4]. Then $\Sigma = \frac{1}{2}[(0,0) + (0,0)] = [[0,0],[0,0]]$ (trivial case with no variance).

# Likely Exam Questions

1. **Explain the difference between classification and regression problems, providing one real-world example of each.**
   - *Answer*: Classification assigns inputs to discrete categories (e.g., spam vs. not spam email), while regression predicts continuous values (e.g., house prices). Classification is like deciding whether to buy or sell a stock (discrete choices), while regression is like predicting the exact future price of that stock (a continuous value). As stated in the Week 5 lecture: "The main difference between classification and regression tasks is the type of output variable, which is discrete for classification but continuous for regression."
2. **What is a linearly separable problem? Why is this concept important for the Perceptron algorithm?**
   - *Answer*: A problem is linearly separable if a straight line (or hyperplane) can perfectly separate all data points of different classes. This is important because the Perceptron algorithm is guaranteed to find a perfect solution (zero training error) if the data is linearly separable, though it may take many iterations. As mentioned in the Week 5 lecture: "If the data is linearly separable, perceptron is guaranteed to find a perfect weight vector, a perfect weight vector meaning that the one that has zero training error."
3. **Describe the Perceptron learning algorithm in your own words. What happens when a data point is correctly classified versus misclassified?**
   - *Answer*: The Perceptron starts with random weights and processes each training example. If a point is correctly classified, no update is made. If misclassified, weights are updated using $w^{(new)} = w^{(old)} + \eta t_n \phi_n$. This moves the decision boundary to better classify that point. As stated in the textbook: "If the training data point is classified correctly, we do not make the above update and proceed to process the next training example."
4. **Given a weight vector w = [0.5, -1.0] and bias $w_0$ = 0.2, determine the class of a data point x = [2, 1] using a linear classifier with threshold 0. Show your calculations.**
   - *Answer*: Compute $a = w^T x + w_0 = (0.5)(2) + (-1.0)(1) + 0.2 = 1.0 - 1.0 + 0.2 = 0.2$. Since $a > 0$, the predicted class is +1. This follows the linear classification approach described in both lectures where "if you give me an X, I compute the score... and then I pick the class which has the highest score."

5. **Why can't we find a closed-form solution for the parameters of logistic regression, unlike linear regression?**
   - *Answer*: The log-likelihood function for logistic regression is non-linear with respect to the parameters, so setting its derivative to zero results in a non-linear system of equations with no analytical solution. As mentioned in the Week 6 lecture: "when you look at this derivative and set it to zero, it doesn't have a closed form solution and that's why we use gradient descent."
6. **Calculate the output of the sigmoid function for a = -1.5 and explain what this value represents in logistic regression.**
   - *Answer*: $\sigma(-1.5) = \frac{1}{1+e^{1.5}} \approx 0.182$. In logistic regression, this represents the predicted probability that the input belongs to class 1. As explained in the Week 6 lecture: "The sigmoid function... takes any real-valued number and squishes it into a value between 0 and 1, perfect for representing probabilities."
7. **What is the key difference between discriminative and generative models for classification? Provide one advantage of each approach.**
   - *Answer*: Discriminative models directly model $P(C_k|x)$ (the probability of class given features), while generative models model $P(x|C_k)$ and $P(C_k)$, then use Bayes' rule. An advantage of discriminative models is they often perform better with limited data since they focus only on the decision boundary. Generative models can generate new data samples and may work better when we understand the data generation process. As stated in the Week 6 lecture: "Discriminative models... directly model $P(C_k|x)$, while generative models... model $P(x|C_k)$ and $P(C_k)$."
8. **Given a binary classification problem with 60 examples in class 1 and 40 in class 2, what is the maximum likelihood estimate for the class prior $P(C_1)$?**
   - *Answer*: $P(C_1) = \frac{60}{60+40} = 0.6$. This follows the maximum likelihood estimate for class priors mentioned in the Week 6 lecture: "for $\phi$, we have seen the class prior before as well, which is also very intuitive it's proportional to the frequency of each class labelled in the data set."
9. **For a 1-dimensional feature space, given 3 data points from class 1: [1], [2], [3], calculate the maximum likelihood estimate of the mean $\mu_1$.**
   - *Answer*: $\mu_1 = \frac{1+2+3}{3} = 2$. This follows the formula from the textbook: "the MLE, the maximum accurate estimate for the parameters of the Gaussian distribution for each class are the empirical ones that you see here" which is simply the sample mean.
10. **Explain why the derivative of the sigmoid function is important for training logistic regression models.**
    - *Answer*: The derivative is needed for gradient-based optimization (like gradient descent). The simple form $\sigma'(a) = \sigma(a)(1 - \sigma(a))$ makes computing gradients efficient. As mentioned in the Week 6 lecture: "Why? Because as you might imagine, later on in the training we are gonna use the training uh we are gonna use a stochastic gradient

descent for training and for a stochastic gradient descent we need the derivative of this guy, OK, with respect to the parameters W."

11. **In the context of the Perceptron algorithm, what does the error function $E(w) = -\sum(w^T\phi_n t_n)$ represent, and why is there a negative sign?**
    - *Answer*: This error function measures how badly misclassified points are being classified. The negative sign ensures that when we minimize this error, we're actually improving classification (since for misclassified points, $w^T\phi_n t_n$ is negative, and we want to make it less negative or positive). As stated in the textbook: "Ideally, we would like to find the parameter vector such that the above error function is zero, i.e. a parameter vector which classifies all the training examples correctly."
12. **Given a logistic regression model with weights $w = [0.8, -0.5]$ and bias $w_0 = 0.1$, calculate the predicted probability for $x = [1, 2]$. Then compute the gradient contribution for this point if the true label is $t = 1$.**
    - *Answer*: First, $a = 0.8(1) + (-0.5)(2) + 0.1 = 0.8 - 1.0 + 0.1 = -0.1$. Then $y = \sigma(-0.1) \approx 0.475$. The gradient contribution is $(t-y)x = (1-0.475)[1, 2] = [0.525, 1.05]$. This follows the gradient formula discussed in the Week 6 lecture: "we basically update to get the next parameter by having the current parameter minus the learning rate times the derivative of the objective function."

# Week 7-8 – Latent Variable Models and Expectation Maximization

## Core Concepts of the Lecture and Material

### Latent Variable Models

Think of latent variable models like trying to understand stock market trends without knowing all the hidden factors that influence prices. Just as you might see stock prices going up or down (the observed data) but not know whether it's due to investor sentiment, economic indicators, or geopolitical events (the latent variables), in machine learning we often have data where some important information is hidden from us. A latent variable model assumes that observed data is generated through a process involving these hidden (latent) variables that we can't directly measure but can infer.

### Supervised vs. Unsupervised Learning

In supervised learning (like what we've seen before), it's like having a stock trading app that shows you both the market conditions (input X) and the resulting stock prices (output Y). But in unsupervised learning, you only see the stock prices (input X) without knowing what caused them - you have to figure out the hidden patterns yourself. Latent variable models sit in this unsupervised space where the "output" or "labels" are hidden from us.

### Clustering Problem

Clustering is like sorting stocks into sectors (tech, healthcare, energy) based on their price movements without being told which sector each belongs to. The goal is to partition data points into groups of similar items. There are two main approaches:

- Hard clustering: Each data point belongs to exactly one cluster (like K-means)
- Soft clustering: Each data point has probabilities of belonging to multiple clusters (like GMM with EM)

### K-means Algorithm

K-means is the "hard clustering" approach. Imagine you're trying to group stocks into sectors based only on their price movements. K-means works in two steps that repeat:

1. Assign each stock to the nearest sector center (cluster center)
2. Recalculate where each sector center should be based on the stocks now assigned to it The key limitation is that each stock must belong to exactly one sector - no partial memberships.

### Gaussian Mixture Models (GMM)

GMM takes a probabilistic approach to clustering. Think of it like modeling stock returns as coming from multiple "hidden" market conditions (bull market, bear market, sideways market). Each condition follows a different Gaussian (normal) distribution. GMM assumes data comes from a mixture of K Gaussian distributions, where:

- $\pi_k$ represents how common each market condition is (mixing coefficient)
- $\mu_k$ represents the average return for each condition (mean)
- $\Sigma_k$ represents how volatile returns are under each condition (covariance)

### Expectation-Maximization (EM) Algorithm

EM is to latent variable models what gradient descent is to optimization problems - a fundamental algorithm. If gradient descent helps you find the lowest point in a valley, EM helps you find the best parameters for your model when some information is missing.

The EM algorithm works in two alternating steps:

- E-step (Expectation): Calculate how likely each data point belongs to each cluster given current parameters. Like estimating how probable it is that today's market movement came from a bull vs. bear market scenario.
- M-step (Maximization): Update the model parameters to maximize the likelihood given these probabilities. Like adjusting your understanding of what defines a bull market based on today's estimated probabilities.

The beauty of EM is that each iteration is guaranteed to improve (or at least not worsen) the model's fit to the data. It's like taking steps that always move you toward better explanations of the stock data, even when you don't know all the hidden factors.

### Document Clustering

Document clustering applies these concepts to text data. Imagine walking into a library with unlabeled books and trying to sort them into categories (science, history, fiction) without reading every page. The "bag-of-words" approach treats each document as just a collection of words, ignoring order - like analyzing a stock's performance based only on which economic indicators were mentioned in news articles about it, not the order they appeared.

In document clustering:

- Each cluster represents a topic (politics, sports, science)
- Each topic has its own word distribution (how likely each word appears)
- We use latent variables to represent which topic generated each document

### Complete Data vs. Incomplete Data

- Complete data: When we know both the documents AND their topics (like having stock prices AND knowing which economic conditions caused them)
- Incomplete data: When we only have the documents (like having stock prices but not knowing the underlying economic conditions)

The challenge is that in real-world scenarios, we only have incomplete data, which makes direct parameter estimation difficult - hence the need for EM.

### Q-function

The Q-function is what we actually maximize in the M-step. It's like creating a "best guess" of the complete data likelihood based on our current understanding, then optimizing as if that guess were true. This makes the optimization tractable where directly optimizing the incomplete data likelihood would be too difficult.

### Challenges with EM

EM can get stuck in local optima - like finding a small valley in the mountains when there's a deeper valley elsewhere. Good initialization is crucial, much like choosing a good starting point for your stock analysis matters. While EM guarantees improvement at each step, it doesn't guarantee finding the absolute best solution.

## Key Formulae

### Gaussian Mixture Model Probability

$$p(x|\theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

- $x$: observed data point
- $\theta$: model parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$
- $\pi_k$: mixing coefficient for cluster $k$ (probability that a random data point belongs to cluster $k$)
- $\mathcal{N}(x|\mu_k, \Sigma_k)$: Gaussian probability density function for cluster $k$
- $K$: total number of clusters

*Example*: For a 2D dataset with two clusters, if $\pi_1 = 0.6$, $\pi_2 = 0.4$, and a data point $x$ has $\mathcal{N}(x|\mu_1, \Sigma_1) = 0.3$ and $\mathcal{N}(x|\mu_2, \Sigma_2) = 0.5$, then $p(x|\theta) = 0.6 \times 0.3 + 0.4 \times 0.5 = 0.18 + 0.2 = 0.38$ .

### Posterior Probability (E-step for GMM)

$$\gamma(z_{nk}) = p(z_k = 1|x_n, \theta) = \frac{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}$$

- $\gamma(z_{nk})$: responsibility - probability that data point $n$ belongs to cluster $k$
- $z_{nk}$: latent variable indicating if data point $n$ belongs to cluster $k$
- $x_n$: $n$-th data point
- $\theta$: current parameter estimates

*Example*: For a data point with $\pi_1 \mathcal{N}(x|\mu_1, \Sigma_1) = 0.18$ and $\pi_2 \mathcal{N}(x|\mu_2, \Sigma_2) = 0.2$, then $\gamma(z_{n1}) = 0.18/(0.18 + 0.2) = 0.474$ and $\gamma(z_{n2}) = 0.2/(0.18 + 0.2) = 0.526$.

### M-step Updates for GMM - Mixing Coefficients

$$\pi_k = \frac{N_k}{N}$$

- $N_k = \sum_{n=1}^{N} \gamma(z_{nk})$: effective number of points assigned to cluster $k$
- $N$: total number of data points

*Example*: If $N = 100$ data points, and the responsibilities for cluster 1 sum to $N_1 = 65.3$, then $\pi_1 = 65.3/100 = 0.653$.

### M-step Updates for GMM - Means

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) x_n$$

- $\mu_k$: updated mean for cluster $k$
- $N_k$: effective number of points in cluster $k$

- $\gamma(z_{nk})$: responsibility of cluster $k$ for data point $n$
- $x_n$: $n$-th data point

*Example*: For cluster 1 with $N_1 = 65.3$, if the weighted sum of data points is $[130.6, 195.9]$, then $\mu_1 = [130.6/65.3, 195.9/65.3] = [2.0, 3.0]$.

### M-step Updates for GMM - Covariances

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^T$$

- $\Sigma_k$: updated covariance matrix for cluster $k$
- $N_k$: effective number of points in cluster $k$
- $\gamma(z_{nk})$: responsibility of cluster $k$ for data point $n$
- $x_n$: $n$-th data point
- $\mu_k$: current mean for cluster $k$

*Example*: For a 1D case with $N_k = 50$, if the weighted sum of squared differences is 100, then $\Sigma_k = 100/50 = 2.0$.

### Document Generation Probability

$$p(d_n, z_n = k|\theta) = \pi_k \prod_{w \in V} (\mu_{kw})^{c_{wn}}$$

- $d_n$: $n$-th document
- $z_n = k$: document $n$ belongs to cluster $k$
- $\pi_k$: probability of cluster $k$
- $\mu_{kw}$: probability of word $w$ in cluster $k$
- $c_{wn}$: count of word $w$ in document $n$
- $V$: vocabulary (dictionary of all words)

*Example*: For a document with words "the cat sat" where "the" appears twice, "cat" once, "sat" once, and for cluster 1: $\pi_1 = 0.4$, $\mu_{1,\text{the}} = 0.1$, $\mu_{1,\text{cat}} = 0.05$, $\mu_{1,\text{sat}} = 0.03$, then $p(d_n, z_n = 1|\theta) = 0.4 \times (0.1)^2 \times 0.05 \times 0.03 = 0.000006$.

### Document Clustering M-step - Word Proportions

$$\mu_{kw} = \frac{\sum_{n=1}^{N} \gamma(z_{nk}) c_{wn}}{\sum_{n=1}^{N} \gamma(z_{nk}) \sum_{w' \in V} c_{w'n}}$$

- $\mu_{kw}$: probability of word $w$ in cluster $k$
- $\gamma(z_{nk})$: responsibility of cluster $k$ for document $n$
- $c_{wn}$: count of word $w$ in document $n$

- $V$: vocabulary

*Example*: For cluster 1 and word "the", if the numerator (weighted sum of "the" counts) is 120 and the denominator (total weighted word count for cluster 1) is 1000, then $\mu_{1,\text{the}} = 120/1000 = 0.12$.

# Likely Exam Questions

1. **Explain the difference between hard clustering and soft clustering with examples.**
   - *Answer*: Hard clustering assigns each data point to exactly one cluster (like K-means where each stock belongs to only one sector). Soft clustering assigns probabilities of belonging to multiple clusters (like GMM with EM where a stock might have 60% probability of being in the tech sector and 40% in the finance sector). As stated in the Week 7 transcript: "In hard clustering, each data point belongs to exactly one cluster; however, the EM algorithm makes soft assignments based on the posterior probabilities."
2. **Why can't we directly maximize the incomplete data log-likelihood for latent variable models?**
   - *Answer*: The incomplete data log-likelihood involves a sum inside a logarithm ($\log \sum$), which makes direct optimization difficult because the logarithm of a sum doesn't simplify nicely. As mentioned in the textbook: "It's very hard to derive the analytical solutions... so we resolve to EM algorithm." The sum inside the log creates a non-convex optimization problem with multiple local optima.
3. **What is the main idea behind the EM algorithm?**
   - *Answer*: EM alternates between two steps: (1) E-step: Compute the expected values of the latent variables given current parameters, and (2) M-step: Maximize the expected complete-data log-likelihood to update parameters. As described in the Week 8 transcript: "The core of EM algorithm... is these two questions... why do we use Q function instead of the log likelihood function as the objective function in MStep?"
4. **Why does the EM algorithm guarantee an increase in log-likelihood at each iteration?**
   - *Answer*: Each EM iteration increases (or at least doesn't decrease) the log-likelihood because the E-step creates a lower bound on the log-likelihood that touches the true log-likelihood at the current parameters, and the M-step maximizes this lower bound. As stated in the textbook: "Each update to the parameters resulting from an E step followed by an M step is guaranteed to increase the log likelihood function."
5. **Given a dataset with 3 points [1, 3, 7] and initial GMM parameters (K=2, $\pi_1=\pi_2=0.5$, $\mu_1=2$, $\mu_2=6$, $\sigma_1=\sigma_2=1$), calculate the responsibilities $\gamma(z_{11})$ and $\gamma(z_{12})$ for the first data point (x=1) in the E-step.**
   - *Answer*: First calculate the Gaussian densities: $\mathcal{N}(1|2,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(1-2)^2}{2}} \approx 0.242$ and $\mathcal{N}(1|6,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(1-6)^2}{2}} \approx 0.000087$. Then $\gamma(z_{11}) = \frac{0.5\times 0.242}{0.5\times 0.242 + 0.5\times 0.000087} \approx 0.9996$ and

$\gamma(z_{12}) \approx 0.0004$. This follows the E-step formula shown in the textbook.

6. **After an E-step, suppose for cluster 1 we have $N_1$ = 45.7 and the weighted sum of data points is [91.4, 137.1]. What would be the updated mean $\mu_1$ in the M-step?**
   - *Answer*: Using the M-step formula for means: $\mu_1 = \frac{1}{N_1} \sum \gamma(z_{n1}) x_n = [91.4/45.7, 137.1/45.7] = [2.0, 3.0]$. This is directly from the M-step update equation for means presented in the textbook.
7. **How is K-means related to GMM with EM?**
   - *Answer*: K-means is a special case of GMM where: (1) the covariance matrices are spherical and equal, (2) the mixing coefficients are equal, and (3) the assignments are hard instead of soft. As stated in the textbook: "The above algorithm is reminiscent of the Kmeans algorithm, with some differences: (i) The only parameters in Kmeans are the cluster means (there are no π and Σ), and (ii) The assignment of data points to clusters are hard in Kmeans where each data point is associated uniquely with one cluster; however, the EM algorithm makes soft assignments."
8. **What is the bag-of-words representation in document clustering?**
   - *Answer*: The bag-of-words representation treats a document as a set of words without considering their order or grammar, only counting word frequencies. As explained in the Week 8 transcript: "In bag of words, we ignore word order and grammar. Only the word counts matter." For example, "Bob went to school" and "school to went Bob" would have identical representations.
9. **Why do we need the EM algorithm for document clustering?**
   - *Answer*: We need EM because the cluster assignments (which topic generated each document) are latent variables that we don't observe. Directly maximizing the incomplete data likelihood is difficult due to the sum inside the logarithm. As stated in the Week 8 transcript: "It's very hard to derive the analytical solutions... so we resolve to EM algorithm." EM provides an iterative approach to handle these hidden cluster assignments.
10. **Given two clusters with $\pi_1$=0.6, $\pi_2$=0.4, and for a document with word counts $c_1$=3 (for word 1) and $c_2$=2 (for word 2), with $\mu_{11}$=0.7, $\mu_{12}$=0.3, $\mu_{21}$=0.2, $\mu_{22}$=0.8, calculate the probability that this document belongs to cluster 1.**

- *Answer*: First calculate $p(d, z_1) = 0.6 \times (0.7)^3 \times (0.3)^2 = 0.6 \times 0.343 \times 0.09 = 0.018522$ and $p(d, z_2) = 0.4 \times (0.2)^3 \times (0.8)^2 = 0.4 \times 0.008 \times 0.64 = 0.002048$. Then $p(z_1|d) = 0.018522/(0.018522 + 0.002048) = 0.9006$. This follows the E-step formula for document clustering shown in the textbook.

11. **What are the potential issues with the EM algorithm?**

- *Answer*: EM can get stuck in local optima (not finding the globally best solution), is sensitive to initialization, and may converge slowly. As mentioned in the Week 8 transcript: "EM can

get stuck in local optima, so please take care of the initialization, meaning it might not find the very best solution globally, but it will always improve, or at least not worsen within each step."

12. **Explain the generative story for document clustering using latent variable models.**

- *Answer*: The generative process is: (1) Choose a cluster/topic k with probability $\pi_k$, (2) For each word position in the document, choose a word from the cluster's word distribution with probability $\mu_k w$. As described in the Week 8 transcript: "First, P(z_n=k) cluster K with probability $\pi_k$, and then for each word slot in the document, generate a word according to the cluster's word distribution." This creates documents where words within a topic tend to co-occur.

# Week 9-10 – Neural Networks: Architecture, Training, and Applications

## Core Concepts of the Lecture and Material

### Neural Network Basics

Neural networks are composed of interconnected processing elements called neurons. Think of these like stock traders in a market—each one processes information (like a stock price) and passes it along to the next set of traders. Each neuron takes multiple inputs (like different market indicators), applies weights (like how much importance to give each indicator), adds a bias term (like a base assumption), and produces an output through an activation function (like a trader's final decision).

The basic structure of a neuron includes:

- Inputs ($x_1$, $x_2$, ..., $x_n$)
- Weights ($w_1$, $w_2$, ..., $w_n$) that determine how important each input is
- A bias term (b) that shifts the activation threshold
- An activation function (f) that introduces nonlinearity

### Activation Functions

Activation functions are crucial for neural networks to model complex patterns—they're like different trading strategies that respond differently to market conditions:

- **Sigmoid function**: Maps inputs to values between 0 and 1. It's like a conservative trader who gradually becomes more confident as evidence accumulates. This function is commonly used in output layers for binary classification problems.
- **Tanh function**: Similar to sigmoid but outputs values between -1 and 1. Think of this as a more sensitive trader who can express both positive and negative confidence.

- **ReLU (Rectified Linear Unit)**: Outputs the input directly if positive, otherwise zero. This is like an aggressive trader who only acts when there's a clear opportunity.

### Neural Network Architecture

Neural networks have layers of neurons:

- **Input layer**: Where data enters the network (like market data coming into a trading desk)
- **Hidden layer(s)**: Intermediate processing layers that extract features from the input (like analysts who process market data)
- **Output layer**: Produces the final prediction (like the final trading decision)

The most basic architecture is the **feed-forward neural network**, where information flows in one direction from input to output with no loops. In these networks, each unit in one layer connects to all units in the next layer (hence "fully connected").

### Forward Propagation

This is the process of computing outputs from inputs. Imagine information flowing through the network like a product moving through a factory assembly line—each station (neuron) processes the product before sending it to the next station. The computation starts with the input layer, moves through hidden layers, and ends at the output layer.

### Training Objectives

The goal is to adjust network parameters to minimize error:

- **Regression**: Minimize sum-of-squares error (like minimizing the difference between predicted and actual stock prices)
- **Binary classification**: Minimize binary cross-entropy (like optimizing the accuracy of predicting whether a stock will go up or down)
- **Multiclass classification**: Minimize multiclass cross-entropy with softmax (like predicting which of several market sectors will outperform)

### Backpropagation Algorithm

This is how neural networks learn from their mistakes. Think of it as a quality control process in a factory where errors at the end are traced back to identify which stations (neurons) need adjustment. The algorithm:

1. Does a forward pass to compute outputs
2. Calculates errors at the output layer
3. Propagates these errors backward through the network
4. Updates weights to reduce future errors

### Parameter Optimization with Gradient Descent

This is the iterative process of adjusting weights to minimize error. Imagine trying to find the

lowest point in a hilly landscape (the error surface) while blindfolded—you take small steps in the direction of the steepest descent. In neural networks, this means:

- Starting with random weights (like random initial trading positions)
- Computing gradients (slopes) to determine which direction reduces error
- Updating weights in the opposite direction of the gradient (adjusting positions to reduce loss)

### Regularization Techniques

These prevent overfitting (when the model becomes too specialized to training data, like a trader who perfectly predicts past market behavior but fails with new data):

- **Weight decay (L2 regularization)**: Adds a penalty term that discourages large weights, like limiting how much capital a trader can allocate to any single position
- **Early stopping**: Halts training when validation error starts increasing, similar to a trader stopping a strategy when it begins underperforming
- **Choosing the right number of hidden units**: Too few (underfitting) is like having too few analysts, while too many (overfitting) is like having so many analysts that they start finding patterns in random noise

### Autoencoders for Unsupervised Learning

These are neural networks that learn to reconstruct their inputs, like a trader who tries to predict tomorrow's market based on today's data. They consist of:

- **Encoder**: Compresses input into a lower-dimensional representation (like distilling key market indicators)
- **Decoder**: Reconstructs the original input from the compressed representation

With linear activation functions, autoencoders perform linear dimensionality reduction equivalent to PCA. With nonlinear activation functions and multiple hidden layers, they can perform nonlinear dimensionality reduction, capturing more complex patterns.

### Self-Taught Learning

This approach leverages both labeled and unlabeled data, like a trader who first studies general market patterns (unlabeled data) before focusing on specific trading strategies (labeled data). The process:

1. Train an autoencoder on unlabeled data to learn good feature representations
2. Use the encoder to transform both labeled and unlabeled data into these learned features
3. Train a classifier on the transformed labeled data

### Visualizing What Hidden Units Learn

To understand what each hidden unit has learned, we can find the input that maximally

activates it. For image data, this reveals features like edges at different positions and orientations—similar to identifying which market indicators most strongly influence a trading decision.

# Key Formulae

### Sigmoid Activation Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z$: Weighted sum of inputs plus bias ($z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$)
- Output: Value between 0 and 1

*Example*: If $z = 0$, $\sigma(0) = \frac{1}{1+e^0} = 0.5$. If $z = 5$, $\sigma(5) \approx 0.993$. This shows how the sigmoid function "squashes" large inputs into a probability-like range.

### Tanh Activation Function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- $z$: Weighted sum of inputs plus bias
- Output: Value between -1 and 1

*Example*: If $z = 0$, $\tanh(0) = 0$. If $z = 2$, $\tanh(2) \approx 0.964$. The tanh function is similar to sigmoid but centered around 0, making it useful for cases where negative outputs are meaningful.

### ReLU Activation Function

$$\mathrm{ReLU}(z) = \max(0, z)$$

- $z$: Weighted sum of inputs plus bias
- Output: $z$ if positive, otherwise 0

*Example*: If $z = -1$, $\mathrm{ReLU}(-1) = 0$. If $z = 3$, $\mathrm{ReLU}(3) = 3$. This simple function helps neural networks learn faster by avoiding the vanishing gradient problem.

### Sum-of-Squares Error (Regression)

$$E = \frac{1}{2}\sum_{n=1}^{N} \|y_n - \hat{y}_n\|^2 + \frac{\lambda}{2}\sum w^2$$

- $y_n$: Target value for training example $n$
- $\hat{y}_n$: Network output for training example $n$
- $\lambda$: Regularization parameter

- $w$: Weight parameters

*Example*: For two data points with targets $y = [2, 3]$ and predictions $\hat{y} = [1.8, 3.2]$, the error would be $\frac{1}{2}[(2 - 1.8)^2 + (3 - 3.2)^2] = 0.04$ before regularization.

### Binary Cross-Entropy Loss

$$E = -\frac{1}{N}\sum_{n=1}^{N}[y_n \log(\hat{y}_n) + (1 - y_n)\log(1 - \hat{y}_n)]$$

- $y_n$: Target value (0 or 1)
- $\hat{y}_n$: Network output (probability between 0 and 1)

*Example*: If true label $y = 1$ and prediction $\hat{y} = 0.9$, loss $= -\log(0.9) \approx 0.105$. If prediction $\hat{y} = 0.5$, loss $= -\log(0.5) = 0.693$—showing how the loss increases as predictions become less confident.

### Softmax Function (Multiclass Classification)

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- $z_i$: Input value for class $i$
- Output: Probability distribution where all values sum to 1

*Example*: For inputs $z = [2, 1, 0]$, the softmax outputs would be $[e^2/(e^2 + e^1 + e^0), e^1/(e^2 + e^1 + e^0), e^0/(e^2 + e^1 + e^0)] \approx [0.665, 0.245, 0.090]$.

### Multiclass Cross-Entropy Loss

$$E = -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} y_{nk}\log(\hat{y}_{nk})$$

- $y_{nk}$: Target value for class $k$ of example $n$ (1 if belongs to class $k$, 0 otherwise)
- $\hat{y}_{nk}$: Predicted probability for class $k$ of example $n$

*Example*: For a 3-class problem where true class is 2 ($y = [0, 1, 0]$) and predictions are $\hat{y} = [0.1, 0.7, 0.2]$, the loss would be $-(0\log(0.1) + 1\log(0.7) + 0\log(0.2)) = -\log(0.7) \approx 0.357$.

### Backpropagation Error Term for Output Layer

$$\delta_i^L = f'(z_i^L)(\hat{y}_i - y_i)$$

- $L$: Output layer
- $f'$: Derivative of activation function
- $\hat{y}_i$: Network output for unit $i$

- $y_i$: Target value for unit $i$

*Example*: For a sigmoid output layer with $z = 0.5$, $\hat{y} = 0.622$, $y = 1$, and $f'(z) = \sigma(z)(1 - \sigma(z)) = 0.235$, the error term $\delta = 0.235 \times (0.622 - 1) = -0.089$.

### Backpropagation Error Term for Hidden Layers

$$\delta_i^l = f'(z_i^l) \sum_j w_{ji}^{l+1} \delta_j^{l+1}$$

- $l$: Current hidden layer
- $w_{ji}^{l+1}$: Weight connecting unit $i$ in layer $l$ to unit $j$ in layer $l + 1$
- $\delta_j^{l+1}$: Error term for unit $j$ in layer $l + 1$

*Example*: For a hidden unit with $z = 0.2$, $f'(z) = 0.2$, and connected to two output units with weights $w = [0.3, -0.4]$ and error terms $\delta = [0.1, -0.2]$, the hidden error term would be $0.2 \times (0.3 \times 0.1 + (-0.4) \times (-0.2)) = 0.022$.

### Weight Update in Gradient Descent

$$w_{ij}^l \leftarrow w_{ij}^l - \eta \frac{\partial E}{\partial w_{ij}^l}$$

- $\eta$: Learning rate
- $\frac{\partial E}{\partial w_{ij}^l}$: Partial derivative of error with respect to weight

*Example*: If current weight $w = 0.5$, learning rate $\eta = 0.1$, and gradient $\frac{\partial E}{\partial w} = 0.3$, the updated weight would be $0.5 - 0.1 \times 0.3 = 0.47$.

## Likely Exam Questions

1. **Explain the key difference between a single perceptron and a neural network, focusing on activation functions and their implications for learning.**
   - *Answer*: A perceptron uses a step function activation which is not differentiable, making it impossible to use gradient-based optimization. Neural networks use continuous, differentiable activation functions (like sigmoid, tanh, or ReLU) that allow for the use of gradient descent and backpropagation to train multi-layer networks. This enables neural networks to learn complex, non-linear decision boundaries, while perceptrons are limited to linear classification. As stated in the textbook (page 5): "The key difference compared to the perceptron, however, is that the neural network uses continuous nonlinearities in the hidden units... This means that the neural network function is differentiable with respect to the network parameters, and this property will play a central role in network training."

2. **Given a neural network with 3 input units, 4 hidden units (with sigmoid activation), and 2 output units (with softmax activation), how many total parameters does the network have? Show your calculation.**
   - *Answer*: For weights between input and hidden layers: 3 inputs × 4 hidden units = 12 weights. For bias terms in hidden layer: 4 biases. For weights between hidden and output layers: 4 hidden units × 2 output units = 8 weights. For bias terms in output layer: 2 biases. Total parameters = 12 + 4 + 8 + 2 = 26. This can be verified from the textbook notation where parameters include weights and biases (page 4).
3. **Why can't we initialize all neural network weights to zero? Explain the problem this would cause for learning.**
   - *Answer*: If all weights are initialized to zero, all hidden units in the same layer will compute identical outputs and gradients during backpropagation. This means all hidden units will learn the same features, effectively reducing the capacity of the network to that of a single hidden unit. The textbook (page 10) states: "If all the parameters start off at identical values, then all the hidden layer units will end up learning the same function of the input... The random initialization serves the purpose of symmetry breaking."
4. **For a binary classification problem with target values of 0 and 1, why is the sigmoid activation function appropriate for the output layer?**
   - *Answer*: The sigmoid function outputs values between 0 and 1, which can be interpreted as probabilities. For binary classification where we predict the probability of class 1, this is ideal. The textbook (page 8) explains: "We can then interpret the network output as the conditional probability $P(y = 1|x)$, with $0 \leq \hat{y} \leq 1$."
5. **Explain the concept of early stopping as a regularization technique. How does it prevent overfitting?**
   - *Answer*: Early stopping halts training when validation error starts to increase, even if training error is still decreasing. This prevents the model from becoming too specialized to the training data. The textbook (page 15) states: "Training can therefore be stopped at the point of smallest error with respect to the validation data set, as indicated in Figure 5.3.3, in order to obtain a network having good generalization performance." The lecture transcript (W10) explains this as monitoring both training and validation errors and stopping when the validation error increases.
6. **Calculate the output of a single neuron with inputs [0.5, 0.2, 0.8], weights [0.3, -0.1, 0.4], bias 0.2, and ReLU activation function.**
   - *Answer*: First compute the weighted sum: $z = (0.5 \times 0.3) + (0.2 \times -0.1) + (0.8 \times 0.4) + 0.2 = 0.15 - 0.02 + 0.32 + 0.2 = 0.65$. Then apply ReLU: $\max(0, 0.65) = 0.65$. This calculation follows the basic neuron structure explained in both the textbook (page 1) and lecture transcripts.
7. **Why is the cross-entropy error function preferred over sum-of-squares for classification problems?**

- *Answer*: Cross-entropy provides stronger error signals when predictions are confident but wrong, leading to faster learning. For example, when the true label is 1 but the prediction is near 0, sum-of-squares gives a small gradient that slows learning, while cross-entropy gives a large gradient. The textbook (page 8) states: "The probabilistic interpretation provides us with a clearer motivation both for the choice of output unit nonlinearity and the choice of error function."

8. **Describe how an autoencoder can be used for dimensionality reduction. What is the difference between a linear autoencoder and a nonlinear autoencoder?**
   - *Answer*: An autoencoder compresses input into a lower-dimensional representation (encoder) and then reconstructs the input from this representation (decoder). A linear autoencoder with one hidden layer performs linear dimensionality reduction equivalent to PCA. A nonlinear autoencoder with multiple hidden layers can capture more complex, nonlinear relationships in the data. The textbook (pages 19-20) explains: "If the hidden units have linear activations functions, then it can be shown that the error function has a unique global minimum, and that at this minimum the network performs a projection onto the k-dimensional subspace which is spanned by the first principal components of the data."
9. **In backpropagation, why do we compute error terms ($\delta$) starting from the output layer and moving backward?**
   - *Answer*: The error at each layer depends on the errors at subsequent layers. Computing from output to input allows us to build on previously calculated errors. The textbook (page 11) explains the intuition: "Given a training datum, we will first run a forward pass to compute all the activations throughout the network... Then, for each node i in layer l, we would like to compute an error term δ_i^l that measures how much that node was responsible for any errors in the output."
10. **How does self-taught learning leverage unlabeled data to improve classification performance?**
   - *Answer*: Self-taught learning first trains an autoencoder on unlabeled data to learn good feature representations. The encoder part of this autoencoder is then used to transform both labeled and unlabeled data into these learned features, which are typically more informative than raw inputs. A classifier is then trained on the transformed labeled data. The textbook (page 24) describes this process: "We can now find a better representation for the inputs... rather than representing the first training example as x^(i), we can feed x^(i) to our autoencoder, and obtain the corresponding vector of activations a."

# Week 11-12 – Machine Learning for Big Data

## Core Concepts of the Lecture and Material

### Learning Curves and Big Data

Learning curves show how model error changes as training data size increases. For low-bias models (like deep neural networks or k-Dependence Bayes), error continues to decrease with more data because they can capture complex patterns. High-bias models (like Naïve Bayes) plateau early because they're too simple to benefit from additional data. Think of this like investing in stocks: low-bias models are diversified portfolios that keep improving as you add more market data, while high-bias models are like single-stock portfolios that can't take advantage of more market information.

### Bias-Variance Tradeoff in Big Data

In small data scenarios, high-bias/low-variance models are preferred to avoid overfitting. But with big data, low-bias models become more valuable because the large dataset reduces variance. This is like how with more historical market data, complex investment models can better predict trends without being overly sensitive to random market fluctuations.

### Model Capacity and Data Size

When you have a large amount of data, you need models with high capacity (complexity) to leverage all the information. Simple models (low capacity) can't take full advantage of big data, just like a basic trading algorithm would miss opportunities in comprehensive market data. The red curve in learning curve graphs represents high-capacity models that keep improving with more data, while blue curves represent low-capacity models that plateau early.

### Map-Reduce Framework

Map-Reduce is a parallel computing paradigm that splits work across multiple machines or cores. The "Map" step processes data subsets in parallel, while the "Reduce" step combines results. Imagine processing stock market data: instead of analyzing all global market data on one computer (which would be slow), you could have different computers analyze data from different regions (Map), then combine the results (Reduce) to get the overall market trend.

### Batch Gradient Descent with Map-Reduce

For gradient descent with large datasets, the Map step calculates partial gradients on data subsets, and the Reduce step combines them to update parameters. This is like having multiple analysts each compute trends for different sectors (Map), then a chief analyst combines these to determine the overall market direction (Reduce).

### K-Means with Map-Reduce

K-Means clustering can be parallelized by having each machine compute sufficient statistics (sums and counts) for its data subset (Map), then combining these to update cluster centers (Reduce). This is analogous to having regional offices calculate local customer spending patterns (Map), which headquarters then aggregates to update overall customer segmentation (Reduce).

### EM Algorithm for GMMs with Map-Reduce

The Expectation-Maximization algorithm for Gaussian Mixture Models can be scaled by parallelizing the E-step (computing responsibilities) across data subsets. The Map step computes sufficient statistics for each partition, and the Reduce step combines them to update model parameters. Think of this as having different teams estimate regional market segments (Map), which are then combined to update the national market segmentation model (Reduce).

### Spark and Hadoop Implementation

Spark is a modern implementation of Map-Reduce that can run on multi-core machines or clusters. It provides faster processing than Hadoop for iterative algorithms like gradient descent by keeping data in memory between iterations. In financial analytics, Spark would allow faster recalculations of portfolio risk as market data streams in.

### Choosing Models for Big Data

Before scaling up an algorithm, check if the learning curve still improves with more data. If a model's learning curve has plateaued (like logistic regression in the textbook example), scaling up won't help. Only scale algorithms where performance continues to improve with more data (like deep neural networks). This is similar to deciding whether to invest in more sophisticated market analysis tools—if your current simple model already captures most of the predictable patterns, further complexity may not be worth the cost.

### Limitations of Big Data Learning

Even with big data, models can't learn beyond their capacity. A high-bias model won't benefit from more data because it can't represent complex patterns. Think of trying to predict complex market trends with only simple moving averages—you can feed it all the historical data in the world, but it still can't capture sophisticated market dynamics.

## Key Formulae

### Batch Gradient Descent Update Rule

$$\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

- $\theta$: model parameters (weight vector)
- $\alpha$: learning rate (controls step size)
- $m$: number of training examples
- $h_\theta(x^{(i)})$: model prediction for input $x^{(i)}$
- $y^{(i)}$: true target value for input $x^{(i)}$
- $x^{(i)}$: input feature vector for the $i$-th training example

*Example: If $\alpha = 0.1$, $m = 4$, and the sum of errors times inputs is $[0.4, 0.8]$, then the parameter update would be $\theta := \theta - 0.1 \times \frac{1}{4} \times [0.4, 0.8] = \theta - [0.01, 0.02]$.*

### Map-Reduce for Batch Gradient Descent

$$\Delta_j^{(k)} = \sum_{i \in \text{subset } k} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{k=1}^{p} \Delta_j^{(k)}$$

- $\Delta_j^{(k)}$: partial gradient for feature $j$ from subset $k$
- $p$: number of partitions/data subsets
- $k$: index of the data subset
- Other variables same as in batch gradient descent

*Example: With 4 data partitions, if the partial gradients for a feature are [0.1, 0.3, 0.2, 0.4], the total gradient would be 1.0. With $\alpha = 0.1$ and $m = 100$, the parameter update would be $\theta_j := \theta_j - 0.1 \times \frac{1}{100} \times 1.0 = \theta_j - 0.001$.*

### K-Means Update Equations

$$S_k = \sum_{i: x^{(i)} \in \text{cluster } k} x^{(i)}$$

$$N_k = |\{i : x^{(i)} \in \text{cluster } k\}|$$

$$\mu_k = \frac{S_k}{N_k}$$

- $S_k$: sum of data points in cluster $k$
- $N_k$: count of data points in cluster $k$
- $\mu_k$: center of cluster $k$
- $x^{(i)}$: $i$-th data point

*Example: If a cluster has points [1,2], [3,4], and [5,6], then $S_k = [9, 12]$, $N_k = 3$, and $\mu_k = [3, 4]$.*

### Map-Reduce for K-Means

$$S_k^{(r)} = \sum_{i \in \text{subset } r, x^{(i)} \in \text{cluster } k} x^{(i)}$$

$$N_k^{(r)} = |\{i \in \text{subset } r : x^{(i)} \in \text{cluster } k\}|$$

$$\mu_k = \frac{\sum_{r=1}^{p} S_k^{(r)}}{\sum_{r=1}^{p} N_k^{(r)}}$$

- $S_k^{(r)}$: sum of points in cluster $k$ from subset $r$
- $N_k^{(r)}$: count of points in cluster $k$ from subset $r$

- $p$: number of partitions

*Example: With 2 partitions, if Partition 1 gives $S_k^{(1)} = [2, 4]$, $N_k^{(1)} = 2$ and Partition 2 gives $S_k^{(2)} = [7, 10]$, $N_k^{(2)} = 3$, then $\mu_k = \frac{[9,14]}{5} = [1.8, 2.8]$.*

### Gaussian Mixture Model Probability

$$p(x) = \sum_{k=1}^{K} \phi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

- $p(x)$: probability density of data point $x$
- $\phi_k$: mixing coefficient for component $k$ (sums to 1)
- $\mathcal{N}(x|\mu_k, \Sigma_k)$: Gaussian probability density function
- $\mu_k$: mean of Gaussian component $k$
- $\Sigma_k$: covariance matrix of Gaussian component $k$

*Example: For a 2-component GMM with $\phi_1 = 0.6$, $\phi_2 = 0.4$, and data point $x$, if $\mathcal{N}(x|\mu_1, \Sigma_1) = 0.3$ and $\mathcal{N}(x|\mu_2, \Sigma_2) = 0.8$, then $p(x) = 0.6 \times 0.3 + 0.4 \times 0.8 = 0.18 + 0.32 = 0.5$.*

## Likely Exam Questions

1. **Explain why low-bias models are particularly valuable when working with big data, using the bias-variance tradeoff perspective.**
   - *Answer*: Low-bias models can capture complex patterns in data but typically have higher variance. With big data, the large sample size reduces variance, allowing these models to achieve better performance. High-bias models, while having lower variance, cannot represent complex patterns regardless of data size. As stated in the textbook (Section 1): "as the size of the training data increases, variance will decrease and hence the most important attribute for accurate learning from big data is low bias."
2. **Describe how the Map-Reduce framework can be applied to batch gradient descent. What is the main computational benefit?**
   - *Answer*: The Map step computes partial gradients on data subsets in parallel across multiple machines/cores. The Reduce step combines these partial gradients to update model parameters. The main benefit is parallelization - instead of processing all data sequentially on one machine, the work is distributed, potentially giving near-linear speedup with more processors. As explained in the textbook (Section 2): "The bulk of the work in gradient descent, was computing the sum over 400 training data points. Now, because each of the four computers can do just a quarter of the work, potentially we can get up to a 4x speed up."
3. **For K-Means clustering, what are the sufficient statistics that need to be computed in the Map step when using Map-Reduce?**

- *Answer*: The sufficient statistics are the sum of data points assigned to each cluster (S_k) and the count of data points in each cluster (N_k). As shown in the textbook (Section 5): "the KMeans algorithm can be described as... for each cluster, calculate the new center as the average of all data points assigned to it... where N_k denotes the size of the set."

4. **Why might it not be beneficial to scale up a machine learning algorithm for a particular problem with big data?**
   - *Answer*: If the learning curve of the model has already plateaued (saturated), adding more data won't improve performance. The textbook (Section 1) states: "we should first ask ourselves whether we gain anything by scaling up our algorithm to handle the full training set... as it can be see, the learning curves of the logistic regression asymptotes (right), while that of the feed-forward network shows the tendency to get improved with more data (left)."
5. **Describe the two main steps of the Expectation-Maximization algorithm for Gaussian Mixture Models.**
   - *Answer*: The E-step computes responsibilities (probabilities of each data point belonging to each Gaussian component). The M-step updates the model parameters (means, covariances, and mixing coefficients) based on these responsibilities. The textbook (Section 5) explains: "the EM for GMMs... [has] two steps: Set and for all to... for to do... where... are sufficient statistics."
6. **Calculate the parameter update for a simple linear regression model with α = 0.01, m = 4, and the following:**
   - $h\theta(x^{(1)}) - y^{(1)} = 0.5$, $x^{(1)} = [1, 2]$
   - $h\theta(x^{(2)}) - y^{(2)} = -0.2$, $x^{(2)}) = [1, 3]$
   - $h\theta(x^{(3)}) - y^{(3)} = 0.8$, $x^{(3)} = [1, 1]$
   - $h\theta(x^{(4)}) - y^{(4)} = -0.1$, $x^{(4)} = [1, 4]$
   - *Answer*: First compute the sum of errors times inputs: [0.5-0.2+0.8-0.1, 1.0-0.6+0.8-0.4] = [1.0, 0.8]. Then update: θ := θ - 0.01 × (1/4) × [1.0, 0.8] = θ - [0.0025, 0.002]. As explained in the textbook (Section 2), this is the batch gradient descent update rule.
7. **What is the main difference between hard clustering and soft clustering? Give an example of each.**
   - *Answer*: Hard clustering assigns each data point to exactly one cluster (e.g., K-Means). Soft clustering assigns probabilities of membership in multiple clusters (e.g., Gaussian Mixture Models using EM). As stated in the lecture transcript: "Hard clustering methods partition the data points into different groups where each data point belongs to only one cluster. In soft clustering, data points could belong to one or more clusters with different degree of membership."
8. **How does the Map-Reduce framework handle the E-step of the EM algorithm for Gaussian Mixture Models?**

- *Answer*: In the Map step, each machine computes the responsibilities and sufficient statistics for its subset of data. In the Reduce step, these partial statistics are combined to compute the complete statistics needed for parameter updates. The textbook (Section 5) explains: "In the Map step, each machine is required to compute the sufficient statistics of the subset of the data assigned to it. In the Reduce step, the computed sufficient statistics for data partitions are combined and model parameters are then updated."

9. **Explain how regularization relates to model capacity when working with big data.**
    - *Answer*: Regularization controls model complexity by adding a penalty term to the error function. With big data, we can use more complex models (high capacity) but still need regularization to prevent overfitting. The textbook (Section 1) notes that "as the size of the training data increases, variance will decrease and hence the most important attribute for accurate learning from big data is low bias." Regularization helps balance this by preventing the model from becoming too complex.
10. **What is the key advantage of Spark over Hadoop for machine learning algorithms?**
    - *Answer*: Spark can keep data in memory between iterations, making it much faster for iterative algorithms like gradient descent, K-Means, and EM. The textbook (Section 2) states: "Nowadays, there exist good open source implementations of Map-Reduce, such as Hadoop or Spark. In the next chapter, we see how to work with Spark to leverage the power of Map-Reduce computation scheme." The lecture transcript also mentions Spark in the context of efficient big data processing.

Here's a study guide to help you prepare for your exam, structured around the specific topics your lecturer provided.

I've analyzed your sample exam PDF1, your lecture notes, and the topics you listed. The exam 2 is **closed book** and seems to focus on a mix of:

- **Conceptual Understanding:** Defining key terms and explaining *why* certain methods are used (e.g., Q1, Q2, Q3a, Q5c)3333333333333333.
- **Algorithmic Processes:** Describing the steps of an algorithm (e.g., Q3b, Q7c)4444.
- **Mathematical Derivations:** Deriving key equations, like for MLE or gradients (e.g., Q4c, Q6b)555.
- **Calculations:** Applying formulas to given numbers (e.g., Q8a, Q8b)6666.

This guide will follow your lecturer's topic list, emphasizing these question types.

---

# 1. Model Complexity, kNN, Polynomial Regression, & Regularization

## 🧠 What to Expect on the Exam

This topic covers the fundamental trade-off in machine learning. Expect questions that ask you to **define overfitting** and **explain how model complexity relates to it**. You'll need to know how different models are controlled:

- **kNN:** How does the value of $k$ affect complexity?
- **Polynomial Regression:** How does the degree $M$ affect complexity?
- **Regularization:** How does the parameter $\lambda$ affect complexity?

The sample exam had a scenario question (Q3) about model selection 7. Be prepared to **critique a flawed workflow** and **describe a correct one** (e.g., using k-fold cross-validation properly to select a parameter, then retraining on all data).

## 📝 Sample Exam Questions

### 1. Theory: Overfitting and Model Complexity

- **Q:** What is overfitting? How does the complexity of a polynomial regression model (the degree $M$) influence the risk of overfitting?
- **A:** Overfitting is when a model learns the noise and random fluctuations in the training data, rather than the true underlying pattern. This results in **low training error** but **high test error**. In polynomial regression, a high degree $M$ makes the model highly complex and flexible. This flexibility allows it to fit the training data perfectly (low bias) but makes it very sensitive to the specific data points (high variance), leading to a high risk of overfitting.

### 2. Theory: kNN Complexity

- **Q:** In the k-Nearest Neighbours (kNN) algorithm, is $k = 1$ a high or low complexity model? Explain your reasoning.
- **A:** $k = 1$ is a **high complexity** model. It creates a very flexible and jagged decision boundary that is highly sensitive to individual data points (noise). This results in low bias but very high variance. Conversely, a large $k$ (e.g., $k = N$) is a very low complexity model that would just predict the majority class for all points, resulting in high bias.

### 3. Theory: L1 vs. L2 Regularization

- **Q:** What is the primary difference between L1 (LASSO) and L2 (Ridge) regularization? What is the practical benefit of L1?
- **A:** Both methods add a penalty term to the error function to discourage large weights.
  - **L2 (Ridge)** uses the squared L2-norm: $\frac{\lambda}{2}||\mathbf{w}||^2$. It shrinks weights *towards* zero but rarely makes them *exactly* zero.

- **L1 (LASSO)** uses the L1-norm: $\lambda\|\mathbf{w}\|_1$. Its penalty term (a diamond shape in 2D) has sharp corners, which can force some weights to become *exactly* zero.
- The practical benefit of L1 is that it performs **automatic feature selection** by effectively removing irrelevant features (setting their weights to zero).

#### 4. Scenario: Model Selection (similar to Q3)

- **Q:** A student wants to choose the best regularization parameter $\lambda$ from $\{0.1, 1.0, 10.0\}$ for a linear regression model. They use 5-fold cross-validation, find that $\lambda = 0.1$ gives the lowest average validation error, and report this error as the final performance estimate. What is the most substantial problem with this workflow?
- **A:** The most substantial problem is **using the validation error as the final performance estimate**. The validation set (and the errors from it) was used to *select* the best $\lambda$. This means the model parameters have been indirectly influenced by the validation data. Reporting this error will be an overly optimistic estimate of performance on unseen data. The correct approach would be to use a separate, final *test set* (that was never used for training or model selection) to get an unbiased performance estimate.

---

## 2. Bias and Variance

### 🧠 What to Expect on the Exam

Your lecturer specifically mentioned the **equation** and the **plot**. You must be able to define bias and variance, write the mathematical decomposition of generalization error, and sketch or describe the trade-off plot. Expect conceptual questions asking you to diagnose a model based on its training and test errors.

### 📝 Sample Exam Questions

#### 1. Theory & Equation: Bias-Variance Decomposition

- **Q:** Define bias and variance. Write the equation for the bias-variance decomposition of expected generalization error.
- **A:**
  - **Bias:** The systematic error of a model. It measures how much the *average* prediction (over all possible training sets) differs from the true underlying function. High bias means the model is too simple (underfitting).
  - **Variance:** The sensitivity of a model to the specific training data. It measures how much the model's predictions *vary* for different training sets. High variance means the model is too complex (overfitting).
  - Decomposition:

$$\text{Generalization Error} = \text{bias}^2 + \text{variance} + \text{irreducible error}$$

Where the irreducible error is the noise $\sigma^2$ in the data itself that no model can eliminate.

### 2. Theory: Diagnosing Model Problems

- **Q:** Your model achieves a 2% error on the training set but a 22% error on the test set. Does this model suffer from high bias or high variance? What are two practical steps you could take to fix it?
- **A:** This model suffers from **high variance** (overfitting). The large gap between the low training error and high test error shows it has memorized the training data but fails to generalize.
  - **Fix 1: Increase regularization** (increase the $\lambda$ parameter) to penalize complexity and simplify the model.
  - **Fix 2: Get more training data.** More data helps reduce variance by making it harder for the model to fit random noise.
  - (Other valid fixes: Remove features, use a simpler model).

### 3. Theory: The Trade-off Plot

- **Q:** Describe the relationship between model complexity, bias, and variance as depicted on the classic bias-variance trade-off plot.
- **A:**
  - As **model complexity increases**:
    - **Bias** steadily **decreases** (the model becomes more flexible and can fit the true function better).
    - **Variance** steadily **increases** (the model becomes more sensitive to the training data noise).
  - The **Total Error** (Bias² + Variance) is U-shaped. It's high for simple models (high bias), high for complex models (high variance), and reaches a minimum "sweet spot" of optimal complexity in the middle.

---

# 3. Gradient Descent (GD, Batch, SGD)

## 🧠 What to Expect on the Exam

You need to know the *differences* between these algorithms and *why* one might be chosen over another. The sample exam (Q6) 9 shows you'll need to *use* the gradient in a derivation, so you should know the general update rule.

## 📝 Sample Exam Questions

### 1. Theory: Comparison of GD Algorithms

- **Q:** Compare Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent (MBGD) in terms of how they compute the gradient and their pros and cons.
- **A:**
  - **BGD:** Computes the gradient using the **entire training set** for a single parameter update.
    - *Pro:* Guaranteed to converge to the (local) minimum. Smooth convergence.
    - *Con:* Extremely slow and computationally infeasible for large datasets.
  - **SGD:** Computes the gradient using **one single training sample** for each update.
    - *Pro:* Very fast per update. The noisy updates can help escape shallow local minima.
    - *Con:* Convergence is very noisy (oscillates) and may never fully converge to the minimum.
  - **MBGD:** A compromise that computes the gradient using a **small batch** (e.g., 32 or 64 samples) for each update.
    - *Pro:* Gets the best of both: stable convergence (like BGD) and computational efficiency (like SGD).
    - *Con:* Adds another hyperparameter (batch size) to tune.

### 2. Theory: SGD for Large Datasets

- **Q:** Why is Stochastic Gradient Descent (SGD) often preferred over Batch Gradient Descent (BGD) for training models on very large datasets?
- **A:** For large datasets (e.g., millions of samples), BGD requires computing the gradient over all samples just to make *one* update, which is computationally prohibitive. SGD makes an update after *every single sample*, allowing it to make much faster progress in the same amount of time. While each update is "noisy," it provides a much faster (though less precise) approximation of the true gradient, which is highly efficient for large, redundant datasets.

---

# 4. Classifiers (Perceptron, Bayes, Discriminative/Generative)

## 🧠 What to Expect on the Exam

This is a core theory section. You *must* know the difference between discriminative and generative models. For Perceptron, the key is the update rule. For Bayes, it's understanding the "Naive" assumption. The sample exam (Q6) focused on logistic regression (a discriminative model), so be ready for derivations related to it.

## 📝 Sample Exam Questions

### 1. Theory: Discriminative vs. Generative

- **Q:** What is the fundamental difference between a discriminative classifier and a generative classifier? Give one example of each.
- **A:**
  - **Discriminative models** directly learn the decision boundary between classes. They model the conditional probability $P(C_k|\mathbf{x})$.
    - *Example:* **Logistic Regression** or Perceptron.
  - **Generative models** learn the distribution of the data *within* each class. They model the class-conditional probability $P(\mathbf{x}|C_k)$ and the class prior $P(C_k)$, then use Bayes' theorem to find $P(C_k|\mathbf{x})$.
    - *Example:* **Naive Bayes** or Gaussian Mixture Models (GMMs).

### 2. Calculation: Perceptron Update Rule

- **Q:** A Perceptron has current weights $\mathbf{w} = [0.2, -0.1]$ and bias $w_0 = 0.1$. It receives a training point $\mathbf{x} = [1, 5]$ with true label $t = -1$. The learning rate is $\eta = 0.5$. Calculate the updated weights $\mathbf{w}^{(new)}$ and bias $w_0^{(new)}$.
- **A:**
  1. **Compute activation:**
     $a = \mathbf{w}^T\mathbf{x} + w_0 = (0.2 \times 1) + (-0.1 \times 5) + 0.1 = 0.2 - 0.5 + 0.1 = -0.2$.
  2. **Get predicted class:** The activation function is a step function. Since $a < 0$, the predicted class $y = -1$.
  3. **Check for misclassification:** The predicted class $y = -1$ matches the true label $t = -1$.
  4. **Update:** The point is **correctly classified**, so **no update is made**.
     - $\mathbf{w}^{(new)} = \mathbf{w} = [0.2, -0.1]$
     - $w_0^{(new)} = w_0 = 0.1$
  - *(Self-correction: Let's assume the label was $t = +1$ to show the update)*
    - If $t = +1$, the point is *misclassified*.
    - The update rule is $\mathbf{w}^{(new)} = \mathbf{w} + \eta t \mathbf{x}$ and $w_0^{(new)} = w_0 + \eta t$.
    - $\mathbf{w}^{(new)} = [0.2, -0.1] + 0.5 \times (+1) \times [1, 5] = [0.2, -0.1] + [0.5, 2.5] = [\mathbf{0.7, 2.4}]$
    - $w_0^{(new)} = 0.1 + 0.5 \times (+1) = \mathbf{0.6}$

### 3. Theory: Bayes Classifier

- **Q:** What is the "Naive" assumption in the Naive Bayes classifier, and why is it useful?
- **A:** The "Naive" assumption is that all the **features (inputs) are conditionally independent given the class**. This means it assumes that the value of one feature (e.g., the word "money") has no bearing on the value of another feature (e.g., the word "free") once we know the class (e.g., "spam"). This assumption is "naive" because it's almost always false in the real world. However, it's useful because it dramatically simplifies the problem: instead of modeling a full, high-dimensional $P(\mathbf{x}|C_k)$, we only need to model $P(x_i|C_k)$ for each feature $i$ separately, which requires much less data.

---

# 5. Latent Variables (K-Means, GMM, EM)

## 🧠 What to Expect on the Exam

This topic was heavily featured in the sample exam (Q7)10. You must know the K-Means algorithm steps. You need to understand the difference between K-Means (hard) and GMM (soft) clustering. For EM, you should be able to describe the **E-step** and **M-step** and understand *why* EM is needed (because of latent variables).

## 📝 Sample Exam Questions

### 1. Theory: Hard vs. Soft Clustering

- **Q:** What is the main difference between hard clustering and soft clustering? Give an example algorithm for each.
- **A:**
  - **Hard Clustering** assigns each data point to *exactly one* cluster. The assignments are binary (you are in cluster A or B, not both).
    - *Example:* **K-Means**.
  - **Soft Clustering** assigns a *probability* (or "responsibility") of membership for each point to *all* clusters. A point might be 60% in cluster A and 40% in cluster B.
    - *Example:* **Gaussian Mixture Models (GMM) trained with EM**.

### 2. Algorithm: K-Means

- **Q:** Describe the K-Means algorithm. Specifically, how are cluster centroids computed and how are points assigned to clusters?
- **A:** K-Means is an iterative algorithm:
  1. **Initialize:** Randomly select $K$ data points to be the initial cluster centroids.
  2. **Repeat until convergence:**

- **Assignment Step:** Assign each data point to the *nearest* cluster centroid (usually based on Euclidean distance).
- **Update Step:** Re-compute the centroid of each cluster by taking the **mean** (average) of all data points assigned to that cluster.

### 3. Algorithm: EM Algorithm

- **Q:** What are the two main steps of the Expectation-Maximization (EM) algorithm for training a GMM?
- **A:**
  1. **E-Step (Expectation):** With the current model parameters (means, covariances, mixing coefficients) fixed, calculate the posterior probabilities (or "responsibilities") $\gamma(z_{nk})$ for each data point $n$. This is the "soft" assignment, representing the probability that point $n$ belongs to cluster $k$.
  2. **M-Step (Maximization):** With the responsibilities $\gamma(z_{nk})$ fixed, re-calculate the model parameters to maximize the expected complete-data log-likelihood. This involves computing new means, covariances, and mixing coefficients using the responsibilities as weights.

---

# 6. Neural Networks (NN)

## 🧠 What to Expect on the Exam

Your list and the sample exam (Q8) 11 are perfectly aligned: this section is about **calculations**. You must be ableto perform a **forward pass** (calculating activations and the final output) and a **backward pass** (using the **chain rule** to "compute the residue" (error signal $\delta$) and **update the weights**).

## 📝 Sample Exam Questions

### 1. Calculation: Forward & Backward Propagation (similar to Q8) 12

- **Q:** Consider a simple network with 2 inputs ($x_1, x_2$), 1 hidden unit ($h_1$), and 1 output unit ($t$). The hidden unit uses a **ReLU** activation function ($\text{ReLU}(a) = \max(0, a)$), and the output unit is linear ($t = a_t$).
  - Weights: $w_1 = 0.5$, $w_2 = -1.0$, $w_3 = 2.0$
  - Biases: $b_h = 0.1$, $b_t = -0.5$
  - Network:
    - $a_h = w_1 x_1 + w_2 x_2 + b_h$
    - $h_1 = \text{ReLU}(a_h)$

    - $a_t = w_3 h_1 + b_t$
    - $t = a_t$
  - Error function: $E = \frac{1}{2}(y - t)^2$ (where $y$ is the true target)
  - Given input $\mathbf{x} = [2, 1]$ and target $y = 3$.

  (a) Perform a forward pass. What is the prediction $t$ and the error $E$?
  (b) Perform a backward pass. Calculate the gradient $\frac{\partial E}{\partial w_1}$.
- A:
  (a) Forward Pass:

  1. $a_h = (0.5 \times 2) + (-1.0 \times 1) + 0.1 = 1.0 - 1.0 + 0.1 = 0.1$
  2. $h_1 = \text{ReLU}(0.1) = 0.1$
  3. $a_t = (2.0 \times 0.1) + (-0.5) = 0.2 - 0.5 = -0.3$
  4. $t = a_t = \mathbf{-0.3}$
  5. $E = \frac{1}{2}(3 - (-0.3))^2 = \frac{1}{2}(3.3)^2 = \frac{1}{2}(10.89) = \mathbf{5.445}$

  (b) Backward Pass (Chain Rule):

  We want $\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial t} \times \frac{\partial t}{\partial a_t} \times \frac{\partial a_t}{\partial h_1} \times \frac{\partial h_1}{\partial a_h} \times \frac{\partial a_h}{\partial w_1}$
  6. $\frac{\partial E}{\partial t} = (t - y) = (-0.3 - 3) = -3.3$
  7. $\frac{\partial t}{\partial a_t} = 1$ (linear activation)
  8. $\frac{\partial a_t}{\partial h_1} = w_3 = 2.0$
  9. $\frac{\partial h_1}{\partial a_h}$: This is the derivative of ReLU. Since $a_h = 0.1 > 0$, the derivative is $1$.
  10. $\frac{\partial a_h}{\partial w_1} = x_1 = 2.0$
  11. **Multiply:** $\frac{\partial E}{\partial w_1} = (-3.3) \times (1) \times (2.0) \times (1) \times (2.0) = \mathbf{-13.2}$

---

# 7. Big Data (MapReduce)

## 🧠 What to Expect on the Exam

Your lecturer's list mentions MapReduce for kNN and EM. Your notes focus on K-Means and GMM/EM, which are more common examples. I'll focus on K-Means and EM. You need to understand the **MapReduce paradigm** and be able to **describe how an algorithm's steps are split** between the Map and Reduce phases.

## 📝 Sample Exam Questions

### 1. Theory: MapReduce Paradigm

- **Q:** Briefly explain the role of the "Map" and "Reduce" steps in the MapReduce framework.
- **A:**

- **Map:** This is the parallel processing step. The "Mapper" takes a chunk of the full dataset and "maps" it to a set of intermediate key/value pairs. This step runs independently on many machines.
- **Reduce:** This is the aggregation step. The "Reducer" collects all intermediate values associated with the *same key* (from all mappers) and "reduces" them into a final output, such as by summing, averaging, or counting.

### 2. Algorithm: MapReduce for K-Means

- **Q:** How can the K-Means algorithm be parallelized using MapReduce?
- **A:** The K-Means update step is easily parallelized:
  - **Map Step:** Each mapper is given a chunk of the data and the *current* cluster centroids. For each data point, it assigns the point to the nearest centroid. It then computes *partial* sums and *partial* counts for each cluster (key). It emits (Key: cluster ID, Value: [partial_sum, partial_count]).
  - **Reduce Step:** The reducer receives all partial sums and counts for a single cluster ID. It sums all the partial sums to get the total sum ($S_k$) and sums all the partial counts to get the total count ($N_k$). It then computes the new centroid $\mu_k = S_k/N_k$ and emits this as the final output.

### 3. Algorithm: MapReduce for EM (GMM)

- **Q:** How can the Expectation-Maximization (EM) algorithm for GMMs be parallelized using MapReduce?
- **A:** The E-step and the M-step's aggregations can be parallelized:
  - **Map Step (E-Step):** Each mapper receives a chunk of data and the *current* GMM parameters ($\mu_k, \Sigma_k, \pi_k$). It computes the responsibilities $\gamma(z_{nk})$ for its local data. It then uses these to compute *partial* sufficient statistics (the weighted sums for the new mean, covariance, and mixing coefficients).
  - **Reduce Step (M-Step):** The reducer receives these partial statistics from all mappers, grouped by cluster ID. It aggregates them (sums them up) to get the *total* sufficient statistics. It then performs the final division to compute the *new* parameters $\mu_k^{(new)}, \Sigma_k^{(new)}, \pi_k^{(new)}$.